

Security in Software Applications

Fuzzy Testing



SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

Test suites and Oracles

To test a system we need

- A **test suite**, i.e., a collection of data
- A **test oracle** which determines whether the test is passed or an error is revealed.

Defining test suites and oracles can be a lot of work

- In the worst case, a test oracle is a long list of specification cases specifying –for each case- exactly what should happen
- In the best case, a test oracle only checks if the application crashes

NOTE: **Crashes are good (for testing)**

Code coverage

Code coverage criteria are used to measure how good a test suite is and they include

- *Statement coverage*: Which (%) program statements have been executed by at least one test?
- *Branch coverage*: Which (%) program branch options have been executed by at least one test?

The fragment

```
void f (int x,y) { if (x>0) {y++;}  y-- ; }
```

needs 1 test case for statement coverage and 2 for branch coverage

Perverse effect of coverage criteria

- High coverage criteria may discourage defensive programming

```
void m(File f) {  
    if <security_check_fails> {  
        log (...);  
        throw (SecurityException);  
    }  
    try{ <the main part of the method> }  
    catch (SomeException) {  
        log (...);  
        <some corrective action>;  
        throw (SecurityException); }  
}
```

If the green defensive code is difficult to trigger in tests, it may be tempting (or required) to remove this code to improve test coverage ...

Annotation as Oracle

Annotations in the code, e.g.

- of C/C++ code in SAL for PREfast
- of Java code in JML for ESC/2 or OpenJML

can be used as test oracle by performing **runtime assertion checking**

❖ Annotations provide a test oracle for free !!

You can test by inputting random data and see if the annotations are violated

Echidna – The Ethereum Smart Contract Fuzzer works in this way!

<https://github.com/crytic/echidna>

- Runtime checking of information flow properties (e.g., SPARTA policies for Android) requires “heavy” instrumentation of the code in order to trace the origin of data *within* the running application, so called **dynamic taint tracking**

Security testing is difficult

Normal testing looks at **correct – wanted** behavior for sensible inputs and some inputs on borderline conditions

Security testing also needs to look for **wrong – unwanted** behavior for really strange input

Normal use of the system is more likely to reveal **functional** problems than **security** problems

- users complain about functional problems
- hackers never complain about security problems

Negative test cases

Thinking about **abuse cases** is a useful way to come up with security tests

- What would an attacker try to do?
- Where could the implementation slip up?

This gives rise to negative test cases: **Tests which
are *supposed* to fail**

iOS goto fail SSL bug

SSLVerifySignedServerKeyExchange function contained:

```
...
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail;
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;
err = sslRawVerify(...);
... other checks ...
fail:
    ... buffer frees (cleanups) ...
    return err
```

Negative test cases

- David Wheeler's 'The Apple gotofail vulnerability: lessons learned' gives a good discussion of this bug and of ways to prevent it, incl. the need for negative test cases

<http://www.dwheeler.com/essays/apple-goto-fail.html>

The FrankenCert test suite provides (broken) certificate chains to test for flaws in the program logic for handling certificate flaws.

[Brubaker et al, Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations, Oakland 2014]

- Note that compiler warnings about dead code (aka unreachable code) could (should?) also have spotted this bug.
- Code coverage requirements on the test suite would also have helped.

Fuzz testing

Fuzzing a.k.a [fuzz testing](#) proposed by B.Miller at Wisconsin in 1988

- People assumed that utility programs could correctly process any input values and these programs were available to all
- Found that they could crash 25-33 % of UNIX utility programs
- Run lots of programs with random input, identify the crashes and correlate the input that caused the crashes.
- *Major problems:* not checking returns, array indices, ...

Fuzzing

it is a very effective security testing technique, largely automated

Basic idea: generate (semi-)automatically random input to see if the application crashes

The original form of fuzzing generated very long input to see if the system crashes with segmentation fault

- What kind of bug would this signal?
 - Buffer overflow problem
- Why a very long input?
 - To make it likely that buffer overruns cross segmentation boundaries

Simple fuzzing

What input would you use for fuzzing ?

- Very long strings
- Empty string
- Max or min values of integers
- Zero or negative integer values
- Depending on what you are fuzzing, consider special values, characters or keywords likely to trigger bugs
 - Null values, newline, end-of-file characters
 - Format string characters
 - Semicolons, slashes, backslashes, quotes
 - Application specific keywords such as halt, DROP TABLES...

Pro and Cons of (black box) fuzzing

Pros

- Little effort
 - Test cases generated automatically as oracle simply looks for crashes
 - Simple fuzzing of C/C++ code binary gives a good idea of robustness of code

Cons

- Will not find all the bugs
- Crashes not easy to analyze, but it is a clear true positive that something is wrong
- For programs with complex input, more work needed for good code coverage

Hence lots of work on 'smarter' fuzzers

Fuzzing web apps

Could a fuzzer detect SQL injections or XSS vulnerabilities?

- For SQL injections monitor database for error messages
 - For XSS see if the website echoes HTML tags in user input
-
- There are various tools to fuzz web applications: Spike proxy, HP Webinspect, AppScan, WebScarab, Wapiti, ...
 - Some fuzzers crawl a website gathering traffic themselves, other fuzzers modify traffic generated by other means

Can we expect false positives/negatives ?

- False negatives due to test cases not hitting the vulnerability
- False positives and negatives due to incorrect test oracles
 - For SQL injection not recognizing some sql database errors (false neg)
 - For XSS signaling quoted echoed responses as XSS (false pos)

Smarter fuzzing

- **Mutation-based** fuzzers
 - Apply **random mutations** to existing valid input
 - E.g., observe network traffic then replay with some modifications
 - More interesting valid input than just random input
- **Generation-based** a.k.a. grammar-based fuzzers
 - Generate **semi-well-formed** input from scratch based on some knowledge of file format or network protocol
 - More work to construct the fuzzer
- **Evolutionary** fuzzers
 - Observe how input is processed **to learn** which mutations are interesting
- **Whitebox** approaches
 - **Analyze the code** to determine interesting inputs

Mutation-based fuzzing

- Supply well-formed input
 - Generate random changes to that input
- No assumption about input
 - Only assumes that variants of well-formed input may cause problems
- Easy to setup and not dependent on program details
- May be strongly biased by the initial input
- Still prone to problems:
 - May rerun the same path over again
 - May be hard to generate inputs for certain paths

Example: `zzuf` <http://sam.zoy.org/zzuf/>

Generation-based fuzzing

- For a given file format or a communication protocol, a generational fuzzer tries to generate files or data packets slightly malformed or hit corner cases in the spec
- Possible start:
 - grammar defining legal input or
 - data format specifications
- Typical things to fuzz
 - Many/all possible values for a specific field
 - Esp undefined values or values Reserved for Future Use (RFU)
 - Incorrect lengths, lengths that are zero, payloads too short/long

Tools to build such fuzzers SNOOZE, SPIKE,
Peach Fuzz, ...

Generation-based fuzzing

- Can be more accurate but at a cost

Pros: More complete search

- Values more specific to the program operations
- Can account for dependencies between inputs

Cons: more work

- Retrieve the specification (is there one?)
- Write the generator - ad hoc

Need to do it for each program and each format

CVEs suggest fuzzing file formats

Microsoft Security Bulletin MS04-028

Buffer Overrun in JPEG Processing (GDI+) Could Allow Code Execution

Impact of Vulnerability: Remote Code Execution

Maximum Severity Rating: Critical

Recommendation: Customers should apply the update immediately

- Root cause: a zero sized comment field, without content.

CVE-2007-0243

Sun Java JRE GIF Image Processing Buffer Overflow Vulnerability

Critical: Highly critical Impact: System access Where: From remote

Description: A vulnerability has been reported in Sun Java Runtime Environment (JRE), which can be exploited by malicious people to compromise a vulnerable system. The vulnerability is caused due to an error when processing GIF images and can be exploited to cause a heap-based buffer overflow via a specially crafted GIF image with an image width of 0. Successful exploitation allows execution of arbitrary code.

Note buffer overflow in (native library of) a memory-safe language

Whitebox fuzzing using symbolic execution

- Basic problem with fuzzing: how can we generate inputs that trigger interesting code executions?
- Fuzzing the following code is unlikely to hit the error case

```
int foo(int x) {  
    y = x+3;  
    if (y==13) abort(); // error  
}
```

The idea behind whitebox fuzzing: if we know the code, then by analysing the code we can find interesting values to try

SAGE (Scalable Automated Guided Execution) is a tool from Microsoft Research that uses symbolic execution of x86 binaries to generate test cases

Whitebox fuzzing using symbolic execution

```
m(int x,y) {  
    x = x + y;  
    y = y - x;  
    if (2*y > 8)  
    { ...  
    }  
    else  
    if (3*x < 10) { ...  
    }  
}
```

Which values should we provide for x and y to trigger execution of the two if-branches?

Symbolic execution

<pre>m(int x,y) { x = x + y; y = y - x; if (2*y > 8) { ... } else if (3*x < 10) { ... } }</pre>	If $x=N$ and $y=M$ x becomes $N+M$ y becomes $M-(N+M) = -N$ if-branch taken if $-2N > 8$, i.e., $N < -4$ Second if-branch taken if $N \geq -4$ and $3(M+N) < 10$
---	--

SMT solvers are tools that can simplify such constraints and produce test data that satisfy them, or prove that they are not satisfiable

This generates test data automatically and with good coverage

Symbolic execution for test generation

- Symbolic execution can be used to automatically generate test cases with good coverage
- *Basic idea*: instead of working with concrete values, variables are given symbolic values and the program is executed with these symbolic values to see if certain points in the program are reached
- Drawback of symbolic execution
 - It is very expensive
 - “state explosion” with loops
- SAGE mitigates this by using a single symbolic execution to generate many test inputs for many execution paths

SAGE example symbolic execution

```
void top(char input[4]) {  
    int cnt = 0;  
    if (input[0] == 'b') cnt++;  
    if (input[1] == 'a') cnt++;  
    if (input[2] == 'd') cnt++;  
    if (input[3] == '!') cnt++;  
    if (cnt >= 3) crash();  
}
```

Which would be interesting test cases and how to find them?

SAGE example

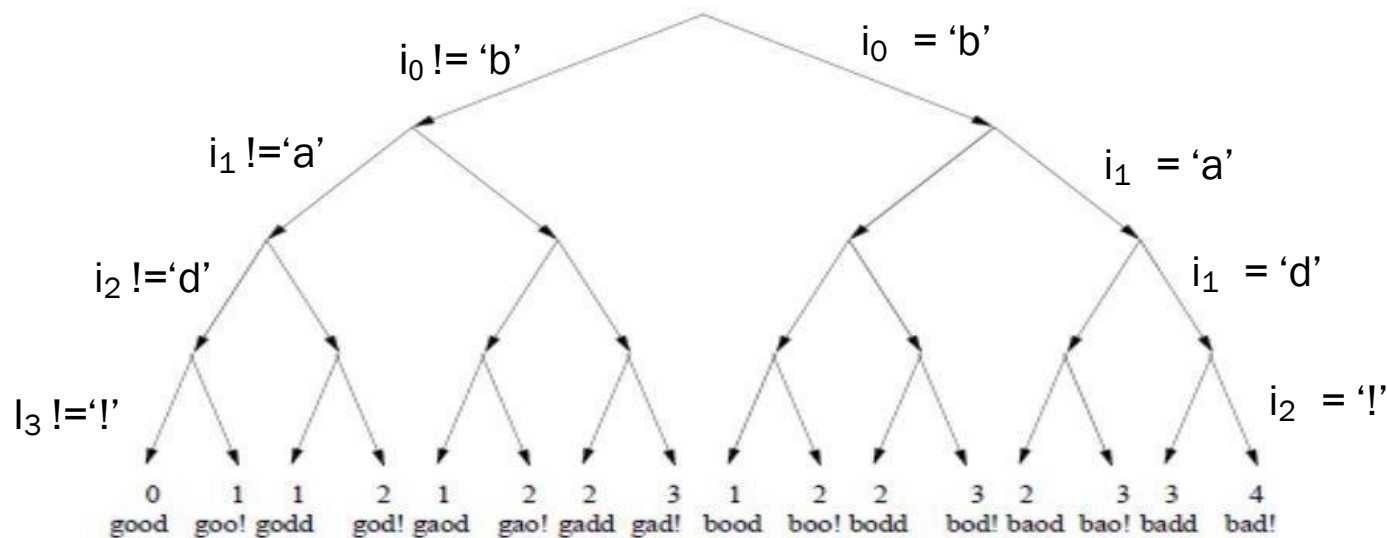
```
void top(char input[4]) {  
    int cnt = 0;  
    if (input[0] == 'b') cnt++;  
    if (input[1] == 'a') cnt++;  
    if (input[2] == 'd') cnt++;  
    if (input[3] == '!') cnt++;  
    if (cnt >= 3) crash();  
}
```

SAGE executes the code for some concrete input, say 'good'

It then collects path constraints $[i_0 \neq \text{'b'}, i_1 \neq \text{'a'}, \dots]$ for an arbitrary symbolic input, say $i_0i_1i_2i_3$

SAGE example

Based on this one execution, combining all these constraints now yealds 16 test cases



Note: the initial execution with input 'good' was not very interesting, but the other ones are

SAGE successful?

SAGE proved successful at uncovering security bugs

- Microsoft Security Bulletin MS07-017 aka CVE-2007-0038 : Critical
- Vulnerabilities of GDI could allow remote code execution
 - Stack-based buffer overflow in the animated cursor code in Microsoft Windows 2000 SP4 through Vista allows remote attackers to execute arbitrary code or cause denial of service ...
- Security vulnerability in parsing the ANI-format. SAGE generated a well formed ANI file triggering the bug, without knowing the ANI format
- First experiments also found bugs in handling a compressed file format, media file formats, and generated 43 test cases to crash Office 2007
- <https://redmondmag.com/articles/2011/07/21/microsoft-credits-sage-for-finding-software-security-flaws.aspx>

Evolutionary Fuzzy Testing

Downside of **generation-based** fuzzing

- Lots of work to write code to do the fuzzing, even if you use tools to generate this code based on some grammar

Downside of **mutation-based** fuzzer

- Chance that random changes in input hits interesting cases is small

afl (**American Fuzzy Lop**) takes an *evolutionary approach* to learn interesting mutations based on measuring code coverage

- Basic idea: if a mutation of the input triggers a new execution path through the code, then it is an interesting mutation and is kept; if not it is discarded
- By trying mutations of the input and observing their effect on code coverage, **afl** can learn what interesting inputs are

afl

- Supports programs written in C/C++/ObjectiveC
- Code instrumented to observe execution paths
 - If source code available, by using a modified compiler
 - If source code is not available by running code in emulator
- Code coverage represented as a 64KB bitmap, where control flow jumps are mapped to changes in this bitmap
 - Different executions could result in the same bitmap but chances are small
- Mutation strategies include: bit flips, incrementing/decrementing integers, using pre-defined interesting integer values such as 0, -1, MAX_INT, ..., deleting/combining/zeroing input blocks

NO need to specify the input format !

Conclusions

- Fuzzing is a good technique to find an interesting class of security flaws
- The bottleneck: how to do smart fuzzing without too much effort
- Current approaches include
 - Whitebox fuzzing based on symbolic execution
 - Evolutionary mutation-based (grey box) fuzzing
 - Black box (mutational and generation)
- Emerging generation of tools that not only tries to find security flaws but also to build exploits for them